



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente
Ingeniería en Desarrollo de Software

Asignatura: **Administración de Bases de Datos**

Unidad #2

Fecha: 25 al 31 de agosto 2025

Tema: Arquitectura del gestor

Introducción:

Este material de lectura tiene como propósito proporcionarte una visión detallada y práctica sobre el funcionamiento interno, las características clave y la arquitectura de los Sistemas de Gestión de Bases de Datos (SGBD), con especial énfasis en plataformas como PostgreSQL. A lo largo del contenido, explorarás los distintos componentes que conforman un DBMS, desde sus estructuras de almacenamiento y memoria, hasta los procesos en segundo plano que garantizan la integridad, disponibilidad y recuperación de los datos. También se abordarán aspectos esenciales como el acceso no procedural, el desarrollo de aplicaciones, el procesamiento de transacciones y los requerimientos para su instalación y puesta en marcha.

Sección 1: Características del DBMS

Un sistema de administración de base de datos (DBMS, por sus siglas en inglés) es un conjunto de componentes que soportan la creación, el uso y el mantenimiento de bases de datos. Estos componentes incluyen tanto herramientas de software como funciones lógicas que permiten a los usuarios interactuar de forma eficiente con los datos.

Inicialmente, los DBMS proporcionaban un eficiente almacenamiento y recuperación de datos, esto incluía tareas básicas como guardar información, consultarla y actualizarla. Sin embargo, gracias a la creciente demanda del mercado y a la innovación tecnológica, los DBMS han evolucionado significativamente. Actualmente ofrecen un amplio rango de características para incorporar, almacenar, diseminar, mantener, recuperar y formatear datos de manera segura, rápida y flexible.

Esta evolución ha implicado un incremento en la complejidad de los DBMS, convirtiéndolos en sistemas robustos que pueden integrarse en múltiples plataformas, desde computadoras personales hasta servidores corporativos y dispositivos móviles. Aprender a utilizar un DBMS en profundidad puede tomar años de estudio y práctica constante, ya que se trata de sistemas dinámicos que están en constante actualización. Por ello, es importante que los profesionales de TI, analistas de datos y desarrolladores mantengan sus conocimientos al día con las nuevas versiones, herramientas y mejores prácticas del sector.

Para mostrar el alcance sobre las funciones que encontrará en los DBMS comerciales, la siguiente tabla resume un conjunto común de funciones. Estas funcionalidades pueden encontrarse en una gran variedad de sistemas, desde DBMS de escritorio como Microsoft Access, hasta sistemas corporativos como Oracle, SQL Server o PostgreSQL.



Función	Descripción
Definición de la base de datos	Lenguaje y herramientas gráficas para definir entidades, relaciones, restricciones de integridad y autorización de privilegios.
Acceso no procedural	Lenguaje y herramientas gráficas para acceder a los datos sin necesidad de código complicado.
Desarrollo de aplicaciones	Herramienta gráfica para desarrollar menús, formularios de captura de datos y reportes; los requerimientos de datos para los formularios y reportes se especifican utilizando un acceso no procedural
Interfase del lenguaje procedural	Lenguaje que combina el acceso no procedural con las capacidades totales de un lenguaje de programación
Procesamiento de transacciones	Mecanismos de control para prevenir la interferencia de usuarios simultáneos y recuperar datos perdidos en caso de una falla
Ajuste de la base de datos	Herramientas para monitorear y mejorar el desempeño de la base de datos

Tabla 2.1- Resumen de las funciones generales de los DBMS.

Definición de datos

La definición de datos es un proceso fundamental en la creación de una base de datos. Este proceso implica identificar y describir las entidades, sus atributos y las relaciones que existen entre ellas. En la mayoría de los Sistemas de Gestión de Bases de Datos (DBMS, por sus siglas en inglés), las entidades se representan mediante tablas, que actúan como contenedores de datos estructurados.

En la mayoría de los DBMS comerciales, las tablas son las estructuras que almacenan los conjuntos de entidades. Cada tabla es una colección organizada de datos que representa una entidad en particular. Una tabla tiene:

- Un renglón de encabezado (el primer renglón), que muestra los nombres de las columnas (también conocidas como campos o atributos), como "ID", "Nombre", "Correo electrónico".
- Un cuerpo (los otros renglones), que contiene los registros o instancias individuales de la entidad. Cada fila representa un objeto específico, como un estudiante con ID 1023, llamado "Ana Torres".

Las relaciones indican las conexiones entre tablas. Por ejemplo, la relación que conecta la tabla estudiante con la tabla inscripción muestra la oferta de cursos tomada por cada estudiante.

La mayoría de los DBMS proporcionan diversas herramientas para definir bases de datos de forma precisa. La principal herramienta utilizada es el Lenguaje de Consulta Estructurado (SQL, por sus siglas en inglés), que es un lenguaje estándar adoptado en la mayoría de los sistemas de gestión de bases de datos.



SQL permite realizar tareas como:

- Crear tablas con la instrucción CREATE TABLE.
- Definir tipos de datos de cada campo, como INT, VARCHAR, DATE, entre otros.
- Establecer restricciones de integridad, como:
 - NOT NULL: el campo no puede quedar vacío.
 - UNIQUE: el valor debe ser único en toda la tabla.
 - CHECK: limita los valores permitidos.
 - PRIMARY KEY: define la llave primaria.
 - FOREIGN KEY: establece una relación con otra tabla.
- Asignar privilegios de acceso mediante GRANT o REVOKE, lo que permite definir quién puede leer, insertar, modificar o eliminar datos.

Además de SQL, muchos DBMS modernos incluyen interfaces gráficas que permiten a los usuarios definir tablas y relaciones de forma visual, arrastrando y soltando componentes sin necesidad de escribir código. Esto facilita el diseño para personas sin experiencia en programación, como analistas de datos o administradores.

Acceso no procedural

Una de las características más poderosas de un DBMS es su capacidad para responder a consultas de los usuarios. Una consulta es una solicitud que busca obtener información específica de la base de datos, por ejemplo, "¿Cuáles son los productos más vendidos en la región occidental?".

En este contexto, el acceso no procedural se refiere a la capacidad de formular dichas consultas sin necesidad de detallar los pasos o algoritmos específicos que deben seguirse para obtener los datos. El usuario especifica el "qué" desea obtener, pero no el "cómo". Este enfoque contrasta con los lenguajes de programación tradicionales, que son procedurales y requieren instrucciones paso a paso (bucles, condicionales, etc.).

Los lenguajes declarativos, como SQL, son ejemplos de acceso no procedural. Por ejemplo, la sentencia SELECT permite recuperar información sin requerir detalles sobre el plan de ejecución, optimización o estructuras internas de la base de datos.

El acceso no procedural tiene múltiples ventajas:

- Reduce drásticamente la cantidad de código necesario (hasta 100 veces menos líneas que un enfoque procedural).
- Facilita el desarrollo de aplicaciones, especialmente para usuarios sin formación técnica avanzada.
- Mejora la productividad y eficiencia, ya que el DBMS se encarga de optimizar internamente la ejecución de las consultas.

Desarrollo de aplicaciones e interfase del lenguaje procedural

Los DBMS modernos van más allá del simple almacenamiento y consulta de datos. Integran herramientas avanzadas para facilitar el desarrollo de aplicaciones completas, que incluyen:

- **Formularios:** permiten ingresar y modificar datos de forma sencilla y visual.
- **Reportes:** presentan la información con un diseño atractivo, ya sea para ser impresa o exportada.



Estas herramientas permiten a los usuarios definir no solo qué datos deben presentarse, sino también cómo deben presentarse e interactuar con ellos. El uso de herramientas gráficas o SQL declarativo simplifica el diseño de interfaces de usuario sin necesidad de escribir código extenso.

Sin embargo, hay casos donde se requiere un mayor nivel de control o eficiencia. En estas situaciones, los DBMS proporcionan interfaces de lenguaje procedural, que permiten:

- Incorporar estructuras de control como bucles, condicionales y procedimientos.
- Crear scripts complejos y funciones almacenadas que integran lógica de negocio directamente en la base de datos.
- Integrar la base de datos con lenguajes de programación externos como Java, Python, C#, PHP, entre otros.

Lenguajes como PL/SQL (en Oracle) o T-SQL (en SQL Server) son ejemplos de lenguajes procedurales embebidos que permiten crear bloques de código que ejecutan instrucciones complejas dentro del DBMS.

Funciones de soporte para las operaciones de base de datos

Procesamiento de transacciones

Una transacción es una secuencia de operaciones que se ejecutan como una unidad lógica de trabajo. Los DBMS garantizan que cada transacción cumpla con las propiedades **ACID**:

- **Atomicidad:** Todas las operaciones de la transacción se completan, o ninguna lo hace.
- **Consistencia:** La base de datos pasa de un estado válido a otro.
- **Aislamiento:** Las transacciones concurrentes no interfieren entre sí.
- **Durabilidad:** Una vez completada, la transacción permanece registrada incluso ante fallos del sistema.

Estas propiedades aseguran que operaciones críticas, como transferencias bancarias, reservas de vuelos o inscripciones académicas, se realicen correctamente y sin corrupción de datos.

Ajuste de la base de datos

El ajuste de rendimiento es crucial para garantizar que la base de datos funcione eficientemente, especialmente cuando hay grandes volúmenes de datos o múltiples usuarios concurrentes. Incluye tareas como:

- Monitoreo de consultas lentas
- Optimización de índices
- Reorganización de tablas
- Distribución eficiente de los datos

Muchos DBMS incorporan herramientas automáticas de análisis y diagnóstico, así como utilerías de mantenimiento que permiten prevenir fallos, liberar espacio y mantener la integridad del sistema.

Tipos de DBMS según su entorno

- **DBMS Corporativos:** diseñados para manejar grandes volúmenes de datos y usuarios. Ejemplos: Oracle, PostgreSQL, Microsoft SQL Server.



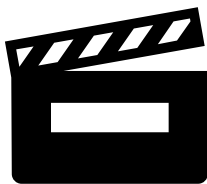
- DBMS de Escritorio: como Microsoft Access o SQLite, ideales para pequeñas empresas o proyectos personales.
- DBMS Embebidos: incorporados dentro de dispositivos o aplicaciones, como en teléfonos móviles, automóviles o electrodomésticos. Tienen requerimientos limitados de recursos y pueden operar sin intervención del usuario.

Funciones de terceros

Además de las funciones nativas que ofrecen los fabricantes de DBMS, existe un amplio ecosistema de software de terceros que amplía y potencia las capacidades de estos sistemas. Estas herramientas, desarrolladas por empresas especializadas, pueden incluir:

- Herramientas de modelado y diseño como ERwin o MySQL Workbench
- Sistemas de respaldo y recuperación avanzados
- Interfaces de integración con software de análisis de datos como Power BI o Tableau
- Plugins de seguridad, auditoría y monitoreo

El ecosistema de herramientas de terceros permite personalizar y escalar las soluciones de bases de datos según las necesidades específicas de cada organización.



Actividad Interactiva #1

Mira el video del link presentado a continuación y escribe en las líneas de abajo un resumen de 4 líneas de lo que consideres más importante.

<https://www.youtube.com/watch?v=HVwS1712nuk>

Sección 2: Estructura de memoria y procesos de la instancia

PostgreSQL es un sistema de gestión de bases de datos relacional (SGBDR) robusto y avanzado, que cuenta con una arquitectura eficiente y modular. Esta arquitectura se compone principalmente de tres elementos fundamentales:

- Memoria compartida
- Procesos en segundo plano
- Estructura del directorio de datos

Comprender cómo interactúan estos componentes es clave para entender el rendimiento, la administración y el mantenimiento de PostgreSQL.

Memoria compartida

La memoria compartida (shared memory) es una porción de la memoria RAM del servidor que es utilizada de forma conjunta por todos los procesos del sistema PostgreSQL. Esta memoria es fundamental porque almacena información crítica que permite un acceso más rápido a los datos y reduce la necesidad de acceder constantemente al disco duro, lo cual sería mucho más lento.



Componentes principales de la memoria compartida:

1. Buffers compartidos (Shared Buffers)

- Estos buffers actúan como una caché de bloques de disco.
- Su propósito principal es reducir el número de accesos directos al disco, acelerando la lectura y escritura de datos.
- Se recomienda asignar aproximadamente el 25% de la memoria total del sistema a los buffers compartidos en servidores dedicados exclusivamente a PostgreSQL.
- Valor por defecto (desde la versión 9.3): 128 MB.

Si una aplicación consulta constantemente una tabla de productos, los bloques que componen esa tabla estarán probablemente en los buffers compartidos. Así, PostgreSQL no necesitará ir al disco cada vez que se consulta.

Este comportamiento se basa en el principio de localidad, que indica si se accede a ciertos datos una vez, es probable que se vuelvan a necesitar pronto.

2. Buffers WAL (WAL Buffers)

- WAL significa Write-Ahead Logging, una técnica utilizada para asegurar la integridad de los datos.
- Estos buffers almacenan temporalmente los cambios que se aplicarán a la base de datos antes de escribirlos en los archivos WAL.
- El archivo WAL se utiliza principalmente para recuperación ante fallos y restauración desde copias de seguridad.

Si se realiza una transacción que actualiza varias filas en una tabla, antes de que estos cambios se escriban en el disco, primero se almacenan en el buffer WAL y luego se registran en el archivo WAL. Esto garantiza que en caso de apagón o fallo del sistema, los cambios puedan recuperarse.

3. Memoria de trabajo (Work Memory)

- Utilizada durante la ejecución de operaciones complejas como:
 - Ordenamientos (ORDER BY)
 - Uniones (JOIN)
 - Operaciones con mapas de bits
- Si la operación requiere más memoria que la asignada, se crean archivos temporales en disco, lo cual disminuye el rendimiento.
- Configuración por defecto (desde PostgreSQL 9.3): 4 MB.

Al realizar un `SELECT ... ORDER BY` sobre una gran cantidad de filas, PostgreSQL usará esta memoria para ordenar los datos antes de devolverlos al cliente.

4. Memoria de trabajo de mantenimiento (Maintenance Work Memory)

- Se utiliza para tareas administrativas como:
 - `VACUUM` (limpieza de registros obsoletos)



- ANALYZE (actualización de estadísticas)
- CREATE INDEX
- ALTER TABLE
- Estas operaciones requieren más recursos que una consulta común.
- Configuración por defecto (desde PostgreSQL 9.4): 64 MB.

Aumentar esta memoria puede acelerar significativamente la creación de índices en bases de datos grandes.

Procesos en segundo plano

PostgreSQL se basa en un modelo de procesos múltiples. Cada uno cumple un rol específico, y todos ellos trabajan de manera coordinada para mantener el sistema estable y eficiente.

Procesos importantes:

1. Proceso de punto de control (Checkpoint)

- Su función es escribir todos los bloques modificados (páginas sucias) desde la memoria al disco.
- También registra un punto de control en el archivo WAL.
- Es clave para la recuperación de la base de datos tras un fallo.
- Cuanto más recientes sean los puntos de control, menos datos hay que recuperar en caso de error.

Si PostgreSQL se detiene inesperadamente, al reiniciarlo, se usará el último punto de control para determinar qué datos necesitan recuperarse.

2. Proceso de escritura en segundo plano (Background Writer)

- Es responsable de escribir bloques modificados (dirty pages) de forma proactiva en el disco, sin esperar a un punto de control.
- A partir de la versión 9.2, este proceso se separó del checkpointer para mejorar el rendimiento general.

3. Escritor de WAL (WAL Writer)

- Su tarea es escribir el contenido del buffer WAL en archivos WAL reales en disco.
- Ayuda a mantener la consistencia de los datos y permite la recuperación en caso de fallos.

4. Archivador (Archiver)

- Solo se activa si se ha configurado la archivación de WAL (archive_mode = on).
- Copia los archivos WAL a un directorio externo o sistema de almacenamiento para respaldo continuo (backup continuo o PITR – Point In Time Recovery).

Ideal para entornos donde se necesita replicación o posibilidad de restaurar el sistema a un estado anterior (como antes de un error de usuario).



5. Registrador de WAL (Logger o WAL Receiver/Sender)

- En algunos contextos se refiere al proceso que recibe y registra los datos WAL en réplicas.
- También puede referirse al process logger, encargado de registrar eventos del sistema y errores en los logs.

Archivos de datos y estructura del directorio de datos

Cuando PostgreSQL se instala e inicializa, crea un conjunto de directorios y archivos que forman la base del clúster de base de datos. Un clúster en PostgreSQL es un conjunto de bases de datos gestionadas por una sola instancia del servidor. Todas estas bases de datos comparten el mismo conjunto de archivos físicos.

Al inicializar un clúster, se crean tres bases de datos:

- template0: Base de datos inmutable usada como plantilla base.
- template1: Plantilla editable usada para crear nuevas bases de datos.
- postgres: Base de datos de uso general que sirve como espacio de trabajo predeterminado.

Las nuevas bases de datos creadas por el usuario son clones de template1. Si se hacen cambios en template1 (por ejemplo, agregar funciones o extensiones), estos estarán disponibles en todas las nuevas bases de datos.

Estructura del directorio de datos (PGDATA)

Este es el directorio raíz donde se almacenan todos los archivos del clúster de bases de datos. Algunos subdirectorios clave:

- global/: Contiene tablas globales compartidas por todas las bases de datos (por ejemplo, roles, tablas del sistema).
- base/: Contiene subdirectorios por cada base de datos, dentro de los cuales se encuentran sus datos físicos.
- pg_notify/: Almacena el estado de los mecanismos de notificación LISTEN/NOTIFY, útiles para comunicación entre procesos.
- PG_VERSION: Archivo que indica la versión de PostgreSQL utilizada en ese clúster.
- postmaster.pid (PID): Archivo que contiene el ID del proceso principal (postmaster). Sirve para asegurar que solo una instancia del servidor esté utilizando ese clúster.

Sección 3: Estructura física de la base de datos.

En PostgreSQL, además de la arquitectura lógica (bases de datos, esquemas, tablas, etc.), existe una estructura física que organiza cómo se almacenan los datos realmente en el sistema de archivos del servidor. Comprender esta estructura es esencial para tareas de administración, respaldo, recuperación y monitoreo del rendimiento.

Directorio de datos (PGDATA)

El directorio principal donde PostgreSQL guarda todos los archivos de una instancia (o clúster) de base de datos se llama comúnmente PGDATA.



Este directorio contiene subdirectorios y archivos que almacenan:

- Las tablas y sus índices
- Los archivos de configuración
- Los registros de transacciones
- La información estadística y de replicación
- Entre otros datos críticos para el funcionamiento de PostgreSQL

Un ejemplo típico de ruta sería:

/pgdata/10/data

Este directorio puede variar dependiendo del sistema operativo, la versión instalada o la configuración personalizada del administrador de base de datos.

Estructura del directorio PGDATA

A continuación, se describe cada uno de los subdirectorios más importantes dentro de PGDATA, junto con su función:

- **base/**
 - Contiene archivos físicos de las tablas y índices de las bases de datos del clúster.
 - Cada base de datos se almacena en un subdirectorio con un identificador numérico.
- **global/**
 - Contiene archivos relacionados con tablas del sistema que son compartidas por todas las bases de datos, como las que gestionan roles, transacciones y configuraciones globales.
- **pg_commit_ts/**
 - Guarda marcas de tiempo asociadas a la confirmación de transacciones.
 - Utilizado para replicación y análisis de cuándo ocurrieron las confirmaciones.
- **pg_dynshmem/**
 - Contiene archivos temporales utilizados para el intercambio dinámico de memoria compartida.
 - Facilita la comunicación entre procesos del servidor de PostgreSQL.
- **pg_logical/**
 - Almacena datos de estado para replicación lógica, una forma avanzada de replicar datos que permite filtrar y transformar los datos replicados.
- **pg_multixact/**
 - Contiene información sobre bloqueos compartidos en filas cuando múltiples transacciones están involucradas.
 - Es fundamental para el manejo de concurrencia y bloqueos complejos.
- **pg_notify/**
 - Guarda el estado de los comandos LISTEN/NOTIFY, una funcionalidad que permite comunicación entre clientes conectados a la misma base de datos.



- **pg_replslot/**
 - Almacena la información de las ranuras de réplica (replication slots), que permiten retener WALs (registros de escritura anticipada) para garantizar que los clientes de réplica no pierdan información.
- **pg_serial/**
 - Guarda datos relacionados con el modo de aislamiento serializable, el más estricto en PostgreSQL, que evita completamente condiciones de carrera en transacciones.
- **pg_snapshots/**
 - Almacena instantáneas (snapshots) de transacciones, necesarias para mantener la consistencia de los datos entre sesiones concurrentes.
- **pg_stat/**
 - Contiene archivos que almacenan estadísticas del sistema, como número de accesos a tablas, índices y tiempos de ejecución.
- **pg_stat_tmp/**
 - Similar a pg_stat, pero contiene archivos temporales de estadísticas. Estos archivos son generados durante el funcionamiento del sistema y suelen ser recreados tras reinicios.
- **pg_subtrans/**
 - Guarda información sobre las subtransacciones, útiles en operaciones que implican transacciones anidadas.
- **pg_tblspc/**
 - Directorio utilizado para el manejo de tablespaces, que permiten distribuir datos en diferentes ubicaciones físicas del disco (por ejemplo, para optimizar rendimiento o espacio).
- **pg_twophase/**
 - Almacena archivos relacionados con transacciones en dos fases (2PC – Two-Phase Commit), útiles en entornos distribuidos donde se necesita asegurar consistencia entre múltiples sistemas.
- **pg_wal/**
 - Directorio crítico que almacena los archivos de registro WAL (Write-Ahead Logging).
 - El WAL garantiza la integridad de los datos permitiendo recuperar operaciones en caso de fallo del sistema.

Sección 4: Requerimientos para la instalación.

Antes de instalar cualquier sistema de gestión de bases de datos (SGBD), es fundamental preparar el entorno del sistema tanto a nivel de hardware como de software. Esto no solo garantiza una instalación exitosa, sino también un rendimiento óptimo del sistema.

Revisión previa a la instalación

- **Requisitos de hardware y software:** Asegúrate de cumplir con los requerimientos mínimos (memoria RAM, espacio en disco, sistema operativo, etc.).
- **Software previo a desinstalar:** En algunos casos, podrían existir conflictos con versiones antiguas del mismo SGBD o con otros servicios en ejecución (como puertos ocupados por otros programas).



- **Verificación del registro de Windows:** En sistemas Windows, es recomendable revisar y limpiar posibles entradas erróneas o duplicadas relacionadas con versiones previas del SGBD.
- **Configuración del entorno del sistema:** Algunos SGBD pueden requerir:
 - Reconfigurar servicios TCP/IP.
 - Asegurar la disponibilidad de ciertos puertos.
 - Modificar tipos de archivos HTML o asociaciones de protocolo para los navegadores, si se accede a través de interfaces web.

Comparación de requerimientos mínimos para la instalación

A continuación, se comparan los requisitos básicos para tres motores de bases de datos comunes (Oracle 11g Express, MySQL 5.1 y PostgreSQL) en sistemas operativos:

Requerimiento	Oracle 11g Express	MySQL 5.1	PostgreSQL
RAM	512 MB	512 MB	512 MB
Memoria Virtual (swap)	1024 MB	1024 MB	1024 MB
Espacio en disco duro	1.5 GB	1.5 GB	1.0 GB
Tamaño máx. de base de datos	4 GB	Sin límite	Sin límite
Sistema Operativo	Windows / Linux / Unix		
Arquitectura del sistema	32/64 bits		
Protocolos de red	TCP/IP, TCP/IP con SSL		

La memoria virtual (también conocida como espacio de intercambio o swap) actúa como una extensión de la memoria RAM usando espacio en disco. Es útil cuando el sistema necesita más memoria de la disponible físicamente.

Regla general:

- Si el sistema tiene menos de 4 GB de RAM, se recomienda tener al menos el doble en memoria virtual.
- Si tiene más de 8 GB de RAM, se puede asignar un tamaño similar al de la RAM.
- En sistemas modernos con suficiente RAM, la memoria virtual se usa poco, pero sigue siendo importante para la estabilidad del sistema.

Sección 4: Instalación del software de BD en modo transaccional.

En la actualidad, las empresas generan grandes volúmenes de datos de manera constante, como resultado de sus múltiples actividades operativas, comerciales y administrativas. Este crecimiento acelerado ha creado la necesidad de implementar plataformas más eficientes para el almacenamiento y análisis de la información. Estas nuevas soluciones deben ser capaces de operar en ambientes que consuman menos recursos, ofrezcan mayor escalabilidad, es decir, que puedan crecer fácilmente según las necesidades y garanticen una alta disponibilidad, lo cual significa que el sistema esté accesible casi todo el tiempo sin interrupciones.



Una solución efectiva frente a esta necesidad es el uso del procesamiento paralelo de datos, lo cual permite que varias operaciones se ejecuten al mismo tiempo, mejorando el rendimiento general del sistema de bases de datos.

Dentro de este contexto, es fundamental comprender el concepto de modo transaccional en una base de datos (BD). Cuando decimos que una base de datos funciona en modo transaccional, nos referimos a que es capaz de ejecutar operaciones de inserción (INSERT), actualización (UPDATE) y eliminación (DELETE) dentro de una transacción. Una transacción es una unidad lógica de trabajo que agrupa varias operaciones que deben ejecutarse de forma completa y coherente.

Este enfoque se basa en el principio de todo o nada: si todas las operaciones dentro de una transacción se ejecutan correctamente, los cambios se confirman (commit); pero si ocurre un error durante alguna de las operaciones, todos los cambios realizados hasta ese punto se deshacen (rollback), y la base de datos regresa al estado anterior a la transacción. De este modo, se garantiza la integridad y consistencia de los datos.

Una transacción está compuesta por una o más instrucciones SQL como INSERT, UPDATE o DELETE, que son agrupadas para ejecutarse juntas. Para que esto sea posible, el software de base de datos debe contar con una funcionalidad conocida como log de transacciones o bitácora de transacciones. Este componente permite registrar cada operación, facilitando tanto la confirmación de los cambios (commit) como la reversión en caso de errores (rollback).

Un software que incluye esta funcionalidad de registro y control de transacciones es considerado un software de base de datos real o completo. En cambio, si no posee esta capacidad, como ocurre con algunos sistemas más antiguos o limitados (por ejemplo, D-Base), se trata más bien de un software que emula el comportamiento de una base de datos, pero que no ofrece todas sus garantías, especialmente en cuanto a confiabilidad y seguridad de los datos.

Características clave del modo transaccional:

- Cada transacción debe concluir de forma completa y coherente, ya sea de manera satisfactoria o con error. No puede quedar en un estado intermedio.
- Se utilizan los siguientes comandos para manejar transacciones:
 - BEGIN TRANS para iniciar una nueva transacción.
 - COMMITTRANS para confirmar los cambios realizados si todo ha salido bien.
 - ROLLBACKTRANS para deshacer todos los cambios si se detecta un error.

Estos comandos forman parte del lenguaje SQL y están disponibles en la mayoría de los sistemas de gestión de bases de datos (SGBD), como MySQL, SQL Server, PostgreSQL, Oracle, entre otros. Aunque la sintaxis puede variar ligeramente según el sistema utilizado, el principio teórico es el mismo para todos.

Sección 5: Variables de ambiente y archivos importantes para la instalación

Al momento de instalar un sistema de gestión de bases de datos (SGBD), no basta con ejecutar un instalador y seguir los pasos del asistente. Existen ciertos elementos clave que permiten que el software funcione correctamente dentro del sistema operativo.



Entre los más importantes se encuentran las variables de ambiente y los archivos de configuración o sistema.

Comprender y configurar adecuadamente estos elementos es esencial para garantizar que la base de datos pueda ejecutarse correctamente, encontrar los recursos que necesita, y que sea accesible para los usuarios y aplicaciones.

Las variables de ambiente son valores definidos en el sistema operativo que proporcionan información necesaria para que los programas se ejecuten correctamente. Funcionan como "indicaciones" para el sistema sobre dónde encontrar archivos, cómo comportarse, qué recursos usar, entre otros.

En el caso de un SGBD, las variables de ambiente ayudan a definir cosas como:

- La ubicación de los archivos ejecutables de la base de datos.
- El directorio donde se encuentran los datos.
- Configuración de idioma, zona horaria, puertos, entre otros.

Ejemplos de variables de ambiente:

Variable	Uso	Ejemplo
PATH	Permite ejecutar comandos del SGBD desde cualquier ubicación en la terminal.	C:\Program Files\PostgreSQL\15\bin
PGDATA	Directorio de datos en PostgreSQL.	C:\Program Files\PostgreSQL\15\data

Archivos importantes para la instalación de un SGBD

Además de las variables de ambiente, los archivos de configuración son fundamentales. Estos archivos contienen instrucciones que determinan cómo se comporta el sistema de base de datos, qué parámetros utiliza y cómo se conecta con otras aplicaciones.

A continuación, se explican algunos archivos clave según el sistema utilizado:

PostgreSQL:

- postgresql.conf: Archivo de configuración principal. Controla el rendimiento, logs, conexiones y más.
- pg_hba.conf: Controla quién puede conectarse a la base de datos y desde dónde (control de acceso).
- postgresqlenv (opcional): Puede usarse para definir variables como PGDATA, PGPORT, etc.

Ubicación típica:

- Linux: /etc/postgresql/15/main/
- Windows: en el directorio de instalación, por ejemplo: C:\Program Files\PostgreSQL\15\data



Importancia de configurar correctamente estas variables y archivos

Una mala configuración puede causar que:

- El servidor de base de datos no se inicie.
- No se puedan realizar conexiones remotas.
- Haya pérdida de rendimiento.
- No se encuentren los datos o se pierda acceso a ellos.

Actividad Interactiva #2

Ejercicio de asociación.

A continuación, verás una lista de términos y una lista de definiciones. Asocia Cada Término con su Definición: Une cada término con la definición correcta.

Términos:

1. Bufferes compartidos
2. Proceso de punto de control (Checkpoint)
3. Memoria de trabajo de mantenimiento
4. Archivador (Archiver)
5. PGDATA
6. WAL (Write-Ahead Logging)



Definiciones:

- a) Área de memoria utilizada por operaciones como VACUUM, ANALYZE, ALTER TABLE y CREATE INDEX, que requieren recursos para el mantenimiento de la base de datos.
- b) Componente encargado de copiar archivos WAL completados a un directorio externo para facilitar la recuperación y el respaldo.
- c) Ruta principal del sistema de archivos donde se almacena toda la información física del clúster de bases de datos.
- d) Mecanismo que asegura que los cambios en los datos sean registrados antes de ser escritos en los archivos físicos, garantizando la durabilidad de las transacciones.
- e) Área de memoria donde se almacenan páginas de datos leídas del disco, con el objetivo de reducir operaciones de entrada/salida (I/O) y acelerar el acceso a datos frecuentes.
- f) Proceso que escribe al disco todas las páginas modificadas en la memoria compartida, asegurando que se mantenga un punto de recuperación consistente ante fallos del sistema.

Instrucciones: Asocia cada término (1-4) con la definición correspondiente (A-D). Y escríbela a continuación.
Ej: "1 – D"

